Contents

1 AI/ML Algorithms

1.1 Detailed Algorithm Specifications for IDS

Comprehensive Algorithm Documentation with Pseudocode and Mathematical Formulations
Last Updated: December 22, 2025

1.2 Table of Contents

1.3 1. CNN-LSTM Hybrid Architecture

1.3.1 1.1 Overview

The CNN-LSTM hybrid combines convolutional layers for spatial feature extraction with LSTM layers for temporal modeling. This architecture is particularly effective for network intrusion detection where traffic patterns exhibit both spatial (packet-level) and temporal (sequence-level) characteristics.

1.3.2 1.2 Mathematical Formulation

CNN Layer:

```
Output[i,j,k] = σ(∑∑ W[m,n,k] * Input[i+m, j+n] + b[k])
```

where: - W: convolution kernel weights - b: bias term - σ: activation function (ReLU) - *: convolution operation

LSTM Cell:

```
Forget gate:    f_t = σ(W_f · [h_{t-1}, x_t] + b_f)
Input gate:     i_t = σ(W_i · [h_{t-1}, x_t] + b_i)
Cell candidate: C̃_t = tanh(W_C · [h_{t-1}, x_t] + b_C)
Cell state:     C_t = f_t ⊙ C_{t-1} + i_t ⊙ C̃_t
Output gate:    o_t = σ(W_o · [h_{t-1}, x_t] + b_o)
Hidden state:   h_t = o_t ⊙ tanh(C_t)
```

where: - σ: sigmoid function - ⊙: element-wise multiplication - W: weight matrices - b: bias vectors

1.3.3 1.3 Complete Architecture

```python
def build_cnn_lstm_model(input_shape, num_classes):
    """
    Build CNN-LSTM hybrid model for intrusion detection
```

```
    Args:
        input_shape: (timesteps, features)
        num_classes: number of attack categories

    Returns:
        compiled Keras model
    """

    model = Sequential()

    # Reshape for Conv1D: (batch, timesteps, features)
    model.add(Input(shape=input_shape))

    # CNN layers for spatial feature extraction
    model.add(Conv1D(filters=64, kernel_size=3, padding='same'))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(Dropout(0.2))

    model.add(Conv1D(filters=128, kernel_size=3, padding='same'))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Dropout(0.2))

    # LSTM layers for temporal modeling
    model.add(LSTM(units=128, return_sequences=True))
    model.add(Dropout(0.3))

    model.add(LSTM(units=64, return_sequences=False))
    model.add(Dropout(0.3))

    # Dense layers for classification
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.5))

    model.add(Dense(num_classes, activation='softmax'))

    # Compile model
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy', 'precision', 'recall']
    )

    return model
```

1.3.4 1.4 Training Algorithm

```python
def train_cnn_lstm(X_train, y_train, X_val, y_val, epochs=50, batch_size=128):
    """
    Train CNN-LSTM model with early stopping and learning rate scheduling
    """

    # Build model
    model = build_cnn_lstm_model(input_shape=(X_train.shape[1], X_train.shape[2]),
                                 num_classes=y_train.shape[1])

    # Callbacks
    early_stop = EarlyStopping(monitor='val_loss', patience=10,
                               restore_best_weights=True)

    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                  patience=5, min_lr=1e-6)

    checkpoint = ModelCheckpoint('best_model.h5', monitor='val_accuracy',
                                 save_best_only=True, mode='max')

    # Train
    history = model.fit(
        X_train, y_train,
        batch_size=batch_size,
        epochs=epochs,
        validation_data=(X_val, y_val),
        callbacks=[early_stop, reduce_lr, checkpoint],
        verbose=1
    )

    return model, history
```

1.3.5 1.5 Pseudocode

```
Algorithm: CNN-LSTM Hybrid Training
Input: Training data X_train, labels y_train
Output: Trained model θ

1. Initialize model parameters θ randomly
2. For epoch = 1 to max_epochs:
3.     Shuffle training data
4.     For each mini-batch B in training data:
5.         # Forward pass
6.         # CNN feature extraction
7.         features = CNN_forward(B, θ_CNN)
8.
9.         # LSTM temporal modeling
```

```
        hidden_states = LSTM_forward(features, θ_LSTM)

        # Classification
        predictions = Softmax(Dense(hidden_states, θ_dense))

        # Compute loss
        loss = CrossEntropy(predictions, y_batch)

        # Backward pass
        gradients = Backpropagate(loss, θ)

        # Update parameters
        θ = Adam_update(θ, gradients, learning_rate)

    # Validation
    val_loss, val_acc = Evaluate(X_val, y_val, θ)

    # Early stopping check
    If val_loss not improving for patience epochs:
        Break

Return θ
```

1.4 2. Transformer Network for IDS

1.4.1 2.1 Overview

Transformers use self-attention mechanisms to capture long-range dependencies in network traffic sequences without the sequential processing limitations of RNNs.

1.4.2 2.2 Mathematical Formulation

Self-Attention:

```
Q = XW_Q    (Query)
K = XW_K    (Key)
V = XW_V    (Value)

Attention(Q, K, V) = softmax(QK^T / √d_k) V
```

Multi-Head Attention:

```
head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)
MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O
```

Position Encoding:

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

1.4.3 2.3 Implementation

```
import tensorflow as tf
from tensorflow.keras import layers

class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super(TransformerBlock, self).__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads,
                                             key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training):
        # Multi-head attention
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)

        # Feed-forward network
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)


class PositionalEncoding(layers.Layer):
    def __init__(self, sequence_length, d_model):
        super(PositionalEncoding, self).__init__()
        self.pos_encoding = self.positional_encoding(sequence_length, d_model)

    def get_angles(self, pos, i, d_model):
        angles = 1 / np.power(10000, (2 * (i // 2)) / np.float32(d_model))
        return pos * angles

    def positional_encoding(self, sequence_length, d_model):
        angle_rads = self.get_angles(
            np.arange(sequence_length)[:, np.newaxis],
            np.arange(d_model)[np.newaxis, :],
            d_model
        )
        # Apply sin to even indices
        angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
```

```
        # Apply cos to odd indices
        angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])

        pos_encoding = angle_rads[np.newaxis, ...]
        return tf.cast(pos_encoding, dtype=tf.float32)

    def call(self, inputs):
        return inputs + self.pos_encoding[:, :tf.shape(inputs)[1], :]


def build_transformer_model(input_shape, num_classes,
                            num_heads=8, ff_dim=256, num_blocks=4):
    """
    Build Transformer model for intrusion detection
    """

    inputs = layers.Input(shape=input_shape)

    # Embedding
    x = layers.Dense(256)(inputs)  # Project to d_model

    # Positional encoding
    x = PositionalEncoding(input_shape[0], 256)(x)

    # Transformer blocks
    for _ in range(num_blocks):
        x = TransformerBlock(embed_dim=256, num_heads=num_heads,
                             ff_dim=ff_dim)(x)

    # Global average pooling
    x = layers.GlobalAveragePooling1D()(x)

    # Classification head
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

1.4.4 2.4 Pseudocode

```
Algorithm: Transformer-based IDS
Input: Traffic sequence X = [x_1, x_2, ..., x_T]
Output: Attack classification

# Token embedding
X_embed = Embedding(X)  # Shape: (T, d_model)

# Positional encoding
For pos = 0 to T-1:
    For i = 0 to d_model-1:
        If i is even:
            PE[pos, i] = sin(pos / 10000^(i/d_model))
        Else:
            PE[pos, i] = cos(pos / 10000^(i/d_model))

X = X_embed + PE

# Multi-layer Transformer
For layer = 1 to N:
    # Multi-head self-attention
    Q, K, V = Linear(X), Linear(X), Linear(X)

    For head = 1 to num_heads:
        attention_scores = softmax(Q_h · K_h^T / √d_k)
        head_output[head] = attention_scores · V_h

    Multi_head = Concat(head_output[1:num_heads]) · W^O
    X = LayerNorm(X + Dropout(Multi_head))

    # Feed-forward network
    FFN = ReLU(X · W_1 + b_1) · W_2 + b_2
    X = LayerNorm(X + Dropout(FFN))

# Classification
pooled = GlobalAveragePooling(X)
output = Softmax(Dense(pooled))

Return output
```

1.5 3. Autoencoder for Anomaly Detection

1.5.1 3.1 Overview

Autoencoders learn to compress and reconstruct normal traffic. Anomalies result in high reconstruction errors, enabling unsupervised detection.

1.5.2 3.2 Mathematical Formulation

Encoder:

$$h = \sigma(W_e \cdot x + b_e)$$

Decoder:

$$\hat{x} = \sigma(W_d \cdot h + b_d)$$

Loss Function:

$$L(x, \hat{x}) = ||x - \hat{x}||^2 = \sum_i (x_i - \hat{x}_i)^2$$

Anomaly Score:

$$\text{score}(x) = ||x - \hat{x}||^2$$

If score(x) > threshold → Anomaly
Else → Normal

1.5.3 3.3 Implementation

```
def build_autoencoder(input_dim, encoding_dim=32):
    """
    Build autoencoder for anomaly detection

    Args:
        input_dim: number of input features
        encoding_dim: dimension of latent representation

    Returns:
        encoder, decoder, autoencoder models
    """

    # Encoder
    input_layer = Input(shape=(input_dim,))
    encoded = Dense(128, activation='relu')(input_layer)
    encoded = Dropout(0.2)(encoded)
    encoded = Dense(64, activation='relu')(encoded)
    encoded = Dropout(0.2)(encoded)
    encoded = Dense(encoding_dim, activation='relu')(encoded)

    encoder = Model(input_layer, encoded)

    # Decoder
    encoded_input = Input(shape=(encoding_dim,))
    decoded = Dense(64, activation='relu')(encoded_input)
    decoded = Dropout(0.2)(decoded)
    decoded = Dense(128, activation='relu')(decoded)
    decoded = Dropout(0.2)(decoded)
    decoded = Dense(input_dim, activation='sigmoid')(decoded)
```

```
    decoder = Model(encoded_input, decoded)

    # Autoencoder
    autoencoder = Model(input_layer, decoder(encoder(input_layer)))

    autoencoder.compile(optimizer='adam', loss='mse')

    return encoder, decoder, autoencoder


def train_autoencoder_unsupervised(X_normal, epochs=100, batch_size=256):
    """
    Train autoencoder on normal traffic only
    """

    autoencoder = build_autoencoder(input_dim=X_normal.shape[1])[2]

    history = autoencoder.fit(
        X_normal, X_normal,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.1,
        callbacks=[EarlyStopping(patience=10)],
        verbose=1
    )

    return autoencoder


def detect_anomalies(autoencoder, X_test, percentile=95):
    """
    Detect anomalies based on reconstruction error

    Args:
        autoencoder: trained model
        X_test: test data
        percentile: threshold percentile for anomaly detection

    Returns:
        predictions: 0 for normal, 1 for anomaly
    """

    # Reconstruct
    X_reconstructed = autoencoder.predict(X_test)

    # Compute reconstruction error
    mse = np.mean(np.power(X_test - X_reconstructed, 2), axis=1)
```

```
    # Determine threshold (e.g., 95th percentile of training errors)
    threshold = np.percentile(mse, percentile)

    # Classify
    predictions = (mse > threshold).astype(int)

    return predictions, mse
```

1.5.4 3.4 Variational Autoencoder (VAE)

```
def sampling(args):
    """Reparameterization trick"""
    z_mean, z_log_var = args
    batch = tf.shape(z_mean)[0]
    dim = tf.shape(z_mean)[1]
    epsilon = tf.random.normal(shape=(batch, dim))
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon


def build_vae(input_dim, latent_dim=32):
    """
    Build Variational Autoencoder
    """

    # Encoder
    encoder_input = Input(shape=(input_dim,))
    x = Dense(128, activation='relu')(encoder_input)
    x = Dense(64, activation='relu')(x)

    z_mean = Dense(latent_dim, name='z_mean')(x)
    z_log_var = Dense(latent_dim, name='z_log_var')(x)

    z = Lambda(sampling, name='z')([z_mean, z_log_var])

    encoder = Model(encoder_input, [z_mean, z_log_var, z], name='encoder')

    # Decoder
    latent_input = Input(shape=(latent_dim,))
    x = Dense(64, activation='relu')(latent_input)
    x = Dense(128, activation='relu')(x)
    decoder_output = Dense(input_dim, activation='sigmoid')(x)

    decoder = Model(latent_input, decoder_output, name='decoder')

    # VAE
    vae_output = decoder(encoder(encoder_input)[2])
    vae = Model(encoder_input, vae_output, name='vae')
```

```
    # VAE loss = reconstruction loss + KL divergence
    reconstruction_loss = tf.reduce_mean(
        tf.reduce_sum(
            tf.keras.losses.binary_crossentropy(encoder_input, vae_output),
            axis=1
        )
    )

    kl_loss = -0.5 * tf.reduce_mean(
        tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var),
                      axis=1)
    )

    vae_loss = reconstruction_loss + kl_loss
    vae.add_loss(vae_loss)
    vae.compile(optimizer='adam')

    return encoder, decoder, vae
```

1.6 4. Generative Adversarial Networks (GANs)

1.6.1 4.1 Overview

GANs generate synthetic attack samples to augment training data, addressing class imbalance and improving model robustness.

1.6.2 4.2 Mathematical Formulation

Generator:

```
G(z) → fake sample
z ~ N(0, I)  (random noise)
```

Discriminator:

```
D(x) → probability that x is real
```

Loss Functions:

```
L_D = -E[log D(x_real)] - E[log(1 - D(G(z)))]
L_G = -E[log D(G(z))]
```

1.6.3 4.3 Implementation

```
def build_generator(latent_dim, output_dim):
    """
    Build generator network
    """
```

```
    model = Sequential([
        Dense(128, input_dim=latent_dim),
        LeakyReLU(alpha=0.2),
        BatchNormalization(),

        Dense(256),
        LeakyReLU(alpha=0.2),
        BatchNormalization(),

        Dense(512),
        LeakyReLU(alpha=0.2),
        BatchNormalization(),

        Dense(output_dim, activation='tanh')
    ])

    return model


def build_discriminator(input_dim):
    """
    Build discriminator network
    """
    model = Sequential([
        Dense(512, input_dim=input_dim),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(256),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(128),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(1, activation='sigmoid')
    ])

    model.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))

    return model


def train_gan(X_real, epochs=10000, batch_size=128, latent_dim=100):
    """
    Train GAN for synthetic attack generation
    """
```

```
    # Build and compile discriminator
    discriminator = build_discriminator(X_real.shape[1])

    # Build generator
    generator = build_generator(latent_dim, X_real.shape[1])

    # Combined model (generator + discriminator)
    discriminator.trainable = False
    gan_input = Input(shape=(latent_dim,))
    x = generator(gan_input)
    gan_output = discriminator(x)
    gan = Model(gan_input, gan_output)
    gan.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))

    # Training loop
    for epoch in range(epochs):
        # Train discriminator
        idx = np.random.randint(0, X_real.shape[0], batch_size)
        real_samples = X_real[idx]

        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        fake_samples = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_samples,
                                                   np.ones((batch_size, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_samples,
                                                   np.zeros((batch_size, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train generator
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        g_loss = gan.train_on_batch(noise, np.ones((batch_size, 1)))

        # Print progress
        if epoch % 1000 == 0:
            print(f"Epoch {epoch}, D Loss: {d_loss:.4f}, G Loss: {g_loss:.4f}")

    return generator, discriminator
```

1.7 5. Ensemble Methods

1.7.1 5.1 Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

```
def train_random_forest(X_train, y_train, n_estimators=100):
    """
    Train Random Forest classifier
    """

    rf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=20,
        min_samples_split=10,
        min_samples_leaf=4,
        max_features='sqrt',
        bootstrap=True,
        n_jobs=-1,
        random_state=42
    )

    rf.fit(X_train, y_train)

    # Feature importance
    feature_importance = pd.DataFrame({
        'feature': feature_names,
        'importance': rf.feature_importances_
    }).sort_values('importance', ascending=False)

    return rf, feature_importance
```

1.7.2 5.2 XGBoost

```
import xgboost as xgb

def train_xgboost(X_train, y_train, X_val, y_val):
    """
    Train XGBoost classifier with hyperparameter tuning
    """

    dtrain = xgb.DMatrix(X_train, label=y_train)
    dval = xgb.DMatrix(X_val, label=y_val)

    params = {
        'objective': 'multi:softmax',
        'num_class': len(np.unique(y_train)),
        'max_depth': 8,
        'learning_rate': 0.1,
        'subsample': 0.8,
        'colsample_bytree': 0.8,
        'min_child_weight': 3,
        'gamma': 0.1,
        'reg_alpha': 0.1,
```

```
        'reg_lambda': 1,
        'eval_metric': 'mlogloss'
    }

    evals = [(dtrain, 'train'), (dval, 'val')]

    model = xgb.train(
        params,
        dtrain,
        num_boost_round=1000,
        evals=evals,
        early_stopping_rounds=50,
        verbose_eval=100
    )

    return model
```

1.8 6. Reinforcement Learning

1.8.1 6.1 Deep Q-Network (DQN)

```
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=10000)
        self.gamma = 0.95    # discount factor
        self.epsilon = 1.0   # exploration rate
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001

        self.model = self._build_model()
        self.target_model = self._build_model()
        self.update_target_model()

    def _build_model(self):
        model = Sequential()
        model.add(Dense(128, input_dim=self.state_size, activation='relu'))
        model.add(Dense(128, activation='relu'))
        model.add(Dense(64, activation='relu'))
        model.add(Dense(self.action_size, activation='linear'))
        model.compile(loss='mse', optimizer=Adam(lr=self.learning_rate))
        return model

    def update_target_model(self):
```

```
        self.target_model.set_weights(self.model.get_weights())

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        if np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
        act_values = self.model.predict(state)
        return np.argmax(act_values[0])

    def replay(self, batch_size=32):
        if len(self.memory) < batch_size:
            return

        minibatch = random.sample(self.memory, batch_size)

        for state, action, reward, next_state, done in minibatch:
            target = reward
            if not done:
                target = reward + self.gamma * np.amax(
                    self.target_model.predict(next_state)[0]
                )

            target_f = self.model.predict(state)
            target_f[0][action] = target

            self.model.fit(state, target_f, epochs=1, verbose=0)

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

1.8.2 6.2 Proximal Policy Optimization (PPO)

```
class PPOAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.gamma = 0.99
        self.epsilon_clip = 0.2
        self.learning_rate = 0.0003

        self.actor = self._build_actor()
        self.critic = self._build_critic()

    def _build_actor(self):
        model = Sequential([
            Dense(128, activation='relu', input_dim=self.state_size),
```

```
            Dense(128, activation='relu'),
            Dense(self.action_size, activation='softmax')
        ])
        model.compile(optimizer=Adam(lr=self.learning_rate), loss='categorical_crossent
        return model

    def _build_critic(self):
        model = Sequential([
            Dense(128, activation='relu', input_dim=self.state_size),
            Dense(128, activation='relu'),
            Dense(1, activation='linear')
        ])
        model.compile(optimizer=Adam(lr=self.learning_rate), loss='mse')
        return model

    def act(self, state):
        probabilities = self.actor.predict(state)[0]
        action = np.random.choice(self.action_size, p=probabilities)
        return action, probabilities[action]

    def train(self, states, actions, rewards, next_states, old_probs):
        # Compute advantages
        values = self.critic.predict(states)
        next_values = self.critic.predict(next_states)

        advantages = rewards + self.gamma * next_values - values

        # Actor loss (PPO clip objective)
        new_probs = self.actor.predict(states)
        ratio = new_probs / (old_probs + 1e-10)
        clipped_ratio = np.clip(ratio, 1 - self.epsilon_clip, 1 + self.epsilon_clip)
        actor_loss = -np.minimum(ratio * advantages, clipped_ratio * advantages)

        # Update actor
        self.actor.fit(states, actions, sample_weight=actor_loss.flatten(), verbose=0)

        # Update critic
        self.critic.fit(states, rewards + self.gamma * next_values, verbose=0)
```

1.9 7. Online Learning

1.9.1 7.1 Incremental Learning

```
from sklearn.linear_model import SGDClassifier

def online_learning_sgd(initial_model, stream_data_generator):
```

```
    """
    Online learning with streaming data

    Args:
        initial_model: pre-trained model
        stream_data_generator: generator yielding (X_batch, y_batch)
    """

    model = SGDClassifier(loss='log', learning_rate='constant', eta0=0.01)

    # Initialize with initial data
    X_init, y_init = next(stream_data_generator)
    model.partial_fit(X_init, y_init, classes=np.unique(y_init))

    # Continuous learning
    for X_batch, y_batch in stream_data_generator:
        # Incremental update
        model.partial_fit(X_batch, y_batch)

        # Evaluate periodically
        if iteration % 100 == 0:
            accuracy = model.score(X_val, y_val)
            print(f"Iteration {iteration}, Accuracy: {accuracy:.4f}")

    return model
```

1.9.2 7.2 Concept Drift Detection

```
def detect_concept_drift(model, X_stream, y_stream, window_size=1000):
    """
    Detect concept drift using Page-Hinkley test
    """

    errors = []
    cumsum = 0
    min_cumsum = 0
    drift_threshold = 50

    for i, (x, y) in enumerate(zip(X_stream, y_stream)):
        # Predict
        y_pred = model.predict([x])[0]
        error = int(y_pred != y)
        errors.append(error)

        # Update cumulative sum
        cumsum += error - np.mean(errors)

        # Page-Hinkley test
```

```
        if cumsum < min_cumsum:
            min_cumsum = cumsum

        drift_magnitude = cumsum - min_cumsum

        if drift_magnitude > drift_threshold:
            print(f"Concept drift detected at sample {i}")
            # Retrain model
            model = retrain_model(X_stream[i-window_size:i],
                                  y_stream[i-window_size:i])
            cumsum = 0
            min_cumsum = 0

    return model
```

1.10 8. Transfer Learning

1.10.1 8.1 Domain Adaptation

```
def transfer_learning_ids(source_model, target_data, freeze_layers=5):
    """
    Transfer learning from source domain to target domain

    Args:
        source_model: pre-trained model on source domain (e.g., NSL-KDD)
        target_data: data from target domain (e.g., CICIDS2017)
        freeze_layers: number of initial layers to freeze
    """

    # Create new model based on source
    target_model = clone_model(source_model)
    target_model.set_weights(source_model.get_weights())

    # Freeze early layers (feature extractors)
    for layer in target_model.layers[:freeze_layers]:
        layer.trainable = False

    # Replace final classification layer
    target_model.pop()  # Remove last layer
    target_model.add(Dense(num_target_classes, activation='softmax'))

    # Fine-tune on target domain
    target_model.compile(optimizer=Adam(lr=0.0001),
                         loss='categorical_crossentropy',
                         metrics=['accuracy'])
```

```
    target_model.fit(X_target_train, y_target_train, epochs=20, batch_size=128)

    return target_model
```

1.11 9. Federated Learning

1.11.1 9.1 FedAvg Algorithm

```
def federated_averaging(global_model, client_data, num_rounds=100):
    """
    Federated Averaging (FedAvg) algorithm

    Args:
        global_model: initial global model
        client_data: list of (X_train, y_train) for each client
        num_rounds: number of communication rounds
    """

    for round_num in range(num_rounds):
        print(f"Round {round_num + 1}/{num_rounds}")

        client_models = []
        client_weights = []

        # Each client trains locally
        for client_id, (X_client, y_client) in enumerate(client_data):
            # Send global model to client
            client_model = clone_model(global_model)
            client_model.set_weights(global_model.get_weights())

            # Local training
            client_model.fit(X_client, y_client, epochs=5, batch_size=32, verbose=0)

            client_models.append(client_model)
            client_weights.append(len(X_client))  # Weight by data size

        # Aggregate models (weighted average)
        total_samples = sum(client_weights)

        for layer_idx in range(len(global_model.layers)):
            if len(global_model.layers[layer_idx].get_weights()) > 0:
                # Weighted average of layer weights
                avg_weights = sum([
                    client_model.layers[layer_idx].get_weights()[0] * weight / total_sa
                    for client_model, weight in zip(client_models, client_weights)
                ])
```

```
            avg_bias = sum([
                client_model.layers[layer_idx].get_weights()[1] * weight / total_sa
                for client_model, weight in zip(client_models, client_weights)
            ])

            global_model.layers[layer_idx].set_weights([avg_weights, avg_bias])

    # Evaluate global model
    if round_num % 10 == 0:
        accuracy = evaluate_global_model(global_model, X_test, y_test)
        print(f"Global Model Accuracy: {accuracy:.4f}")

return global_model
```

1.12 Conclusion

This document provides comprehensive algorithm specifications for all AI/ML components in the proposed IDS framework. Each algorithm includes mathematical formulations, implementation details, and pseudocode for clarity and reproducibility. These algorithms form the foundation of the hybrid, adaptive, and distributed intrusion detection system.

Key Takeaways: - CNN-LSTM combines spatial and temporal feature learning - Transformers capture long-range dependencies via self-attention - Autoencoders enable unsupervised anomaly detection - GANs generate synthetic attack samples for data augmentation - Ensemble methods improve robustness and accuracy - Reinforcement learning enables adaptive threat response - Online learning handles concept drift - Transfer learning enables cross-domain knowledge sharing - Federated learning preserves privacy in distributed scenarios

---

Next: Refer to `datasets.md` for data preparation details and `evaluation_metrics.md` for performance assessment methodology.